

گزارش فاز اول پروژه تبدیل زبان Hash به Promela

فاطمه باقریان – امیرحسین پیرنژاد

تیر ماه ۱۴۰۵

فهرست مطالب

۳	۱	مقدمه
۳	۱.۱	ارتباط با نظریه اتوماتا و زبان‌ها
۳	۲.۱	زیرمجموعه زبان Hash پشتیبانی شده
۳	۳.۱	محدودیت‌های پیاده‌سازی
۴	۴.۱	نحوه عملکرد کل سیستم
۵	۲	توضیح کلاس Main
۵	۱.۲	خواندن ورودی
۵	۲.۲	تولید Parser و Lexer
۵	۳.۲	بررسی خطاهای نحوی
۶	۴.۲	بررسی معنایی
۶	۵.۲	تولید کد Promela
۶	۶.۲	ذخیره خروجی
۷	۳	چرا از Visitor استفاده شده است؟
۷	۱.۳	مقایسه Visitor و Listener
۷	۲.۳	دلایل اصلی انتخاب Visitor
۸	۳.۳	نمونه‌های عملی از مزایای Visitor
۹	۴	طراحی (HashToPromela) Translator
۹	۱.۴	ساختار کلی کلاس
۹	۲.۴	متد visitProgram - نقطه ورود اصلی
۱۰	۳.۴	متد visitVarDecl - ترجمه تعریف متغیرها
۱۰	۴.۴	ترجمه عبارات (Expressions) در HashToPromela
۱۰	۱.۴.۴	روش کلی ترجمه
۱۱	۲.۴.۴	سلسله‌مراتب ترجمه عبارات
۱۱	۳.۴.۴	نحوه اتصال متدها
۱۱	۴.۴.۴	عملگرهای پشتیبانی‌شده
۱۱	۵.۴.۴	مدیریت عملگرهای خاص
۱۲	۶.۴.۴	مزایای روش Visitor
۱۲	۵.۴	متد visitAssignment - ترجمه انتساب
۱۲	۶.۴	متد visitIfStmt - ترجمه دستور شرطی
۱۲	۷.۴	متد visitWhileStmt - ترجمه حلقه while
۱۳	۸.۴	متد visitForStmt - ترجمه حلقه for
۱۳	۹.۴	متد visitTryStmt - ترجمه ساختار try-catch

۱۴	۵	توابع کمکی مهم
۱۴	۱.۵	mapType - نگاشت انواع داده
۱۴	۲.۵	guardDivisionByZero - محافظت از تقسیم بر صفر
۱۴	۳.۵	findZeroDivisors - یافتن مقسوم‌علیه‌های صفر
۱۵	۴.۵	expandPower - تبدیل توان
۱۵	۵.۵	indent - فرمت‌بندی خروجی
۱۵	۶.۵	exitLoopFlags / enterLoopFlags - مدیریت پرچم‌های حلقه
۱۶	۶	نتیجه‌گیری و جمع‌بندی

فصل ۱

مقدمه

۱.۱ ارتباط با نظریه اتوماتا و زبان‌ها

- یک برنامه در حال اجرا را می‌توان به عنوان یک State Machine در نظر گرفت - هر وضعیت حافظه یک State است و هر دستور، یک Transition.
- خواصی که روی برنامه بررسی می‌شوند، با استفاده از Linear Temporal Logic (LTL) بیان می‌شوند.
- هر فرمول LTL معادل یک Büchi Automaton است - یک ماشین حالت محدود که روی کلمات نامتناهی کار می‌کند.
- ابزار SPIN این اتوماتا را با مدل سیستم ترکیب کرده و تمام حالت‌های ممکن را جستجو می‌کند.
- پس آنچه در این پروژه انجام می‌دهیم، عملاً همان نظریه‌ای است که در کلاس خوانده‌ایم را به صورت عملی و قابل لمس پیاده‌سازی می‌کنیم.

۲.۱ زیرمجموعه زبان Hash پشتیبانی شده

زیرمجموعه مورد نظر شامل موارد زیر است:

- متغیرهای از نوع adad و boole
- دستور شرطی age/bood/vagarna
- حلقه‌های ta و baraye به همراه shekan و edame
- ساختار gereftar/emtehan به صورت پایه (یک بلوک catch بدون تودرتو)
- عملگرهای محاسباتی و مقایسه‌ای تعریف شده در پروژه اول

۳.۱ محدودیت‌های پیاده‌سازی

- دستور shekan تنها از حلقه‌ای که مستقیماً داخل آن است خارج می‌شود، نه از حلقه‌های بیرونی.
- دستور edame اجرای بدنه فعلی را متوقف کرده و به شرط حلقه بازمی‌گردد.

- برای gereftar/emtehan، تنها یک بلوک gereftar پشتیبانی می‌شود و تودرتوی بلوک‌های emtehan در این پروژه لازم نیست.
- کلاس‌ها و متدها جزو این زیرمجموعه نیستند.

۴.۱ نحوه عملکرد کل سیستم

فرآیند اجرای پروژه به صورت زیر است:



در این معماری، ابتدا کد ورودی توسط Lexer به توکن‌ها تبدیل می‌شود. سپس Parser بر اساس گرامر تعریف شده، درخت نحوی (Parse Tree) را تولید می‌کند. پس از اطمینان از عدم وجود خطای نحوی، Semantic Checker قواعد معنایی (مانند تعریف متغیرها قبل از استفاده) را بررسی می‌کند. در نهایت، Visitor درخت را پیمایش کرده و کد معادل Promela تولید می‌کند.

فصل ۲

توضیح کلاس Main

کلاس Main نقطه شروع اجرای برنامه است و وظیفه هماهنگی بین بخش‌های مختلف را بر عهده دارد.

۱.۲ خواندن ورودی

```
CharStream input = CharStreams.fromPath(Paths.get(args[0]));
```

این خط فایل ورودی که به عنوان آرگومان خط فرمان دریافت شده است را به جریان کاراکتری تبدیل می‌کند. این جریان سپس توسط *Lexer* پردازش می‌شود. استفاده از *Paths.get* و *CharStreams.fromPath* روش استاندارد و امن برای خواندن فایل در JAVA است.

۲.۲ تولید *Parser* و *Lexer*

```
HashLexer lexer = new HashLexer(input);
CommonTokenStream tokens = new CommonTokenStream(lexer);
HashParser parser = new HashParser(tokens);
```

- *Lexer*: جریان کاراکتری را به توکن‌های معنادار (مانند کلیدواژه‌ها، عملگرها، شناسه‌ها و اعداد) تبدیل می‌کند.
- *TokenStream*: جریان توکن‌ها را به صورتی که *Parser* بتواند از آن استفاده کند، سازماندهی می‌کند.
- *Parser*: بر اساس گرامر تعریف شده در فایل *Hash.g4*، توکن‌ها را تحلیل کرده و درخت نحوی انتزاعی تولید می‌کند.

۳.۲ بررسی خطاهای نحوی

```
if (parser.getNumberOfSyntaxErrors() > 0) {
    System.out.println("Syntax errors found. Semantic
        checking skipped.");
    return;
}
```

در صورت وجود خطای نحوی، ادامه پردازش متوقف می‌شود. این کار از تولید خروجی ناقص یا اشتباه جلوگیری می‌کند. خطاهای نحوی ممکن است شامل استفاده از کلیدواژه‌های اشتباه، قرارگیری نادرست پرانتزها، یا ساختارهای غیرمجاز باشند.

۴.۲ بررسی معنایی

```
۱ SemanticChecker checker = new SemanticChecker();  
۲ checker.collectDef(tree);  
۳ checker.SemanticCheck(tree);
```

در این مرحله، دو عملیات مهم انجام می‌شود:

۱. **جمع‌آوری تعاریف:** collectDef تمام تعاریف متغیرها، توابع و کلاس‌ها را از درخت استخراج کرده و در یک جدول نماد (Symbol Table) ذخیره می‌کند.

۲. **بررسی معنایی:** SemanticCheck قواعد زیر را بررسی می‌کند:

- استفاده از متغیرها قبل از تعریف
- تطابق نوع داده در عملیات‌ها
- تعریف تکراری متغیرها
- استفاده صحیح از ساختارهای شرطی و حلقه‌ها
- و

اگر خطای معنایی وجود داشته باشد، پیام‌های خطا نمایش داده می‌شوند و برنامه ادامه نمی‌یابد.

۵.۲ تولید کد Promela

```
۱ HashToPromela translator = new HashToPromela();  
۲ String outputCode = translator.visit(tree);
```

در این مرحله، درخت نحوی توسط Visitor پیمایش می‌شود و برای هر گره، کد معادل Promela تولید می‌گردد. استفاده از Visitor به ما امکان می‌دهد تا دقیقاً کنترل کنیم که برای هر نوع گره چه کدی تولید شود.

۶.۲ ذخیره خروجی

```
۱ Path outputDir = Paths.get("src/codes");  
۲ Path outputFile = outputDir.resolve("output.pml");  
۳ Files.writeString(outputFile, outputCode);
```

کد تولید شده در فایل output.pml در پوشه src/codes ذخیره می‌شود. این مسیر استاندارد باعث می‌شود که خروجی به راحتی قابل دسترسی و استفاده در فازهای بعدی باشد.

فصل ۳

چرا از Visitor استفاده شده است؟

۱.۳ مقایسه Visitor و Listener

در این پروژه از الگوی Visitor به جای Listener استفاده شده است. جدول زیر مقایسه جامعی بین این دو الگو ارائه می‌دهد:

ویژگی	Visitor	Listener
مقدار بازگشتی	دارد (می‌تواند هر نوع داده‌ای برگرداند)	ندارد (void)
کنترل پیمایش	کامل (خودتان تصمیم می‌گیرید چه گره‌هایی را ببینید)	محدود (پیمایش خودکار توسط ANTLR)
کاربرد اصلی	تولید کد، تبدیل، تحلیل عمیق	تحلیل ساده، جمع‌آوری اطلاعات
وابستگی به گرامر	کم (هر گره یک متد مجزا دارد)	زیاد (همه در یک متد)
پیچیدگی پیاده‌سازی	بیشتر (نیاز به مدیریت پیمایش)	کمتر (پیمایش خودکار)
قابلیت توسعه	بالا (افزودن عملیات جدید بدون تغییر گره‌ها)	متوسط

جدول ۱.۳: مقایسه الگوهای Visitor و Listener

۲.۳ دلایل اصلی انتخاب Visitor

۱. **تولید مقدار بازگشتی:** مهم‌ترین دلیل، نیاز به تولید کد Promela به عنوان خروجی است. در Visitor، هر متد می‌تواند یک رشته (یا هر نوع داده دیگری) برگرداند، در حالی که Listener خروجی ندارد.

۲. **کنترل کامل بر پیمایش:** در Visitor، خودمان تصمیم می‌گیریم که چه گره‌هایی را بازدید کنیم و به چه ترتیبی. این ویژگی برای ترجمه دقیق دستورات Hash به Promela ضروری است، زیرا ترتیب اجرای دستورات و ساختارهای کنترلی باید به درستی حفظ شود.

۳. **بهبود قابلیت نگهداری:** با استفاده از Visitor، هر قانون گرامر یک متد مجزا دارد که کار اصلاح و توسعه را بسیار ساده‌تر می‌کند. اگر بخواهیم یک دستور جدید اضافه کنیم، فقط یک متد جدید می‌نویسیم.

۴. **مدیریت بهتر Context:** در فرآیند ترجمه، نیاز به نگهداری وضعیت‌هایی مانند برچسب‌های حلقه‌ها یا پرچم‌های استثنا داریم. Visitor با استفاده از متغیرهای عضو کلاس یا پشته‌ها، این کار را به سادگی امکان‌پذیر می‌کند.

۵. انتخابی بازدید از گره‌ها: در Visitor، می‌توانیم از بازدید برخی گره‌ها صرف‌نظر کنیم یا گره‌های خاصی را چندین بار بازدید کنیم، در حالی که در Listener همه گره‌ها دقیقاً یک بار بازدید می‌شوند.

۳.۳ نمونه‌های عملی از مزایای Visitor

- مدیریت تقسیم بر صفر: در متدهای مختلف (مانند visitWhileStmt)، می‌توانیم تصمیم بگیریم که قبل از شرط اصلی حلقه، یک شرط اضافی برای بررسی تقسیم بر صفر اضافه کنیم.
- مدیریت حلقه‌های تو در تو: با استفاده از پشته continueLabels، می‌توانیم به راحتی برچسب‌های حلقه‌های تو در تو را مدیریت کنیم. هر بار که وارد یک حلقه جدید می‌شویم، برچسب آن را به پشته اضافه می‌کنیم و پس از خروج، آن را حذف می‌کنیم.
- مدیریت استثناها: در ساختار try-catch، با استفاده از پرچم‌های استثنا که روی پشته قرار می‌گیرند، می‌توانیم رفتار استثنا را در Promela شبیه‌سازی کنیم.

فصل ۴

طراحی (HashToPromela) Translator

کلاس HashToPromela مسئول اصلی تبدیل کد Hash به Promela است. این کلاس از HashBaseVisitor<String> ارث‌بری کرده و برای هر گره از درخت نحوی، یک رشته خروجی تولید می‌کند.

۱.۴ ساختار کلی کلاس

• متغیرهای سراسری اضافه شده:

- divByZero: برای بررسی ویژگی Safety (جلوگیری از تقسیم بر صفر)
- inLoop و exitLoop: برای بررسی ویژگی‌های Liveness
- endReached: برای بررسی ویژگی Reachability
- activeLoopCount: برای مدیریت حلقه‌های تو در تو

• پشته‌ها:

- continueLabels: ذخیره برچسب‌های حلقه‌ها برای دستور edame
- exceptionFlags: ذخیره پرچم‌های استثنا برای ساختار try-catch
- catchLabels: ذخیره برچسب‌های بلوک‌های catch

• شمارنده‌ها:

- loopCounter: تولید برچسب‌های یکتا برای حلقه‌ها
- exceptionCounter: تولید شناسه‌های یکتا برای استثناها

۲.۴ متد visitProgram - نقطه ورود اصلی

این متد مهم‌ترین متد کلاس است و ساختار کلی فایل خروجی را تولید می‌کند:

۱. تعریف متغیرهای کمکی: در ابتدای خروجی، تمام متغیرهای کمکی که برای بررسی خواص سیستم نیاز هستند، تعریف می‌شوند:

```

1      bool divByZero = false;
2      bool inLoop = false;
3      bool exitLoop = false;
4      bool endReached = false;
5      int activeLoopCount = 0;

```

۲. پردازش متغیرهای سراسری: اعلان‌های سطح بالا که از نوع تعریف متغیر هستند، شناسایی شده و به صورت متغیرهای سراسری در Promela تعریف می‌شوند. این کار باعث می‌شود که متغیرها در کل برنامه قابل دسترسی باشند.

۳. تعریف بدنه اصلی: سایر گره‌های سطح بالا (دستورات، توابع، کلاس‌ها) درون یک پروسه فعال با نام main قرار می‌گیرند، همچنین متغیرهای سراسری در این قسمت مقدار دهی می‌شوند:

```

1      active proctype main() {
2          \\ body
3      }

```

۴. افزودن برچسب پایان: در انتهای بدنه، برچسب endReachedLabel و دستور تنظیم endReached = true اضافه می‌شود تا بتوان ویژگی Reachability را بررسی کرد.

۳.۴ متد visitVarDecl - ترجمه تعریف متغیرها

این متد وظیفه تبدیل اعلان متغیرها را بر عهده دارد:

۱. نگاشت نوع: نوع داده Hash به نوع معادل در Promela تبدیل می‌شود:

- $\text{int} \rightarrow \text{adad}$
- $\text{bool} \rightarrow \text{boole}$
- سایر انواع (مانند matn, harf) پشتیبانی نمی‌شوند

۲. مدیریت عملگرهای افزایش/کاهش: اگر عبارت اولیه شامل ++x یا x++ باشد، به صورت زیر ترجمه می‌شود (به طور مشابه برای --):

- برای ++x: ابتدا مقداردهی با مقدار فعلی، سپس افزایش
- برای x++: ابتدا افزایش، سپس مقداردهی با مقدار جدید

۳. بررسی تقسیم بر صفر: اگر عبارت شامل عملگر تقسیم باشد، تابع guardDivisionByZero فراخوانی می‌شود تا از تقسیم بر صفر جلوگیری کند.

۴.۴ ترجمه عبارات (Expressions) در HashToPromela

۱.۴.۴ روش کلی ترجمه

در کلاس HashToPromela، ترجمه عبارات با استفاده از الگوی Visitor و به صورت بازگشتی انجام می‌شود. به این معنی که هر عبارت بزرگ به زیرعبارت‌های کوچکتر تقسیم شده و هر کدام به صورت مستقل ترجمه می‌شوند، سپس نتایج با هم ترکیب می‌شوند.

۲.۴.۴ سلسله مراتب ترجمه عبارات

ترجمه عبارات از بالا به پایین و به ترتیب زیر انجام می شود:

visitExpr()	← Entry Point
visitLogicalOrExpr()	← Logical OR ()
visitLogicalAndExpr()	← Logical AND (&&)
visitEqualityExpr()	← Equality (==, !=)
visitRelationalExpr()	← Relational (<, >, <=, >=)
visitAdditiveExpr()	← Addition/Subtraction (+, -)
visitMultiplicativeExpr()	← Multiplication/Division (*, /, %)
visitPowerExpr()	← Power (**)
visitUnaryExpr()	← Unary (!, -, ++, --)
visitPostfixExpr()	← Postfix (++ , --)
visitPrimaryExpr()	← Primary (Numbers, Variables, Parentheses)

۳.۴.۴ نحوه اتصال متدها

هر متد Visitor یک رشته برمی گرداند و برای زیرعبارتها، متد visit مناسب را فراخوانی می کند. به عنوان مثال، متد visitAdditiveExpr برای ترجمه (a + b) به این صورت عمل می کند:

۱. ابتدا visit(ctx.multiplicativeExpr(0)) را صدا می زند که به "a" می رسد

۲. عملگر "+" را دریافت می کند

۳. سپس visit(ctx.multiplicativeExpr(1)) را صدا می زند که به "b" می رسد

۴. در نهایت "a + b" را برمی گرداند

۴.۴.۴ عملگرهای پشتیبانی شده

معادل Promela	عملگرهای Hash	دسته
+، -، *، /، (ضرب متوالی)	+، -، *، /، **	حسابی
همان عملگرها	<، <=، >، >=، !=	مقایسه ای
!، ، &&	!، ، &	منطقی
x = x - 1، x = x + 1	x --، x++، --x، ++x	یکانی

جدول ۱.۴: عملگرهای پشتیبانی شده

۵.۴.۴ مدیریت عملگرهای خاص

• توان (**): با تابع expandPower به ضرب متوالی تبدیل می شود:

$$x * x * x \rightarrow x^{**3} -$$

• افزایش/کاهش (++)، (--): ترتیب ارزیابی پیشوند و پسوند حفظ می شود:

$$- \rightarrow x++ \text{ ابتدا مقدار فعلی، سپس افزایش}$$

$$- \rightarrow ++x \text{ ابتدا افزایش، سپس مقدار جدید}$$

• تقسیم بر صفر: با تابع guardDivisionByZero مدیریت می شود

۶.۴.۴ مزایای روش Visitor

- کد تمیز و خواناتر: هر عملگر در متد جداگانه
- توسعه‌پذیری بالا: اضافه کردن عملگر جدید با اضافه کردن یک متد
- مدیریت خودکار بازگشت: ANTLR خودش تشخیص می‌دهد کدام متد صدا زده شود

۵.۴ متد visitAssignment - ترجمه انتساب

عملگرهای انتساب ساده و مرکب را پشتیبانی می‌کند:

```
1      switch (op) {
2          case "=": return left + " = " + right;
3          case "+=": return left + " = " + left + " + " + right;
4          case "-=": return left + " = " + left + " - " + right;
5          case "*=": return left + " = " + left + " * " + right;
6          case "/=": return left + " = " + left + " / " + right;
7      }
```

برای عملگر /=، بررسی تقسیم بر صفر اضافه می‌شود.

۶.۴ متد visitIfStmt - ترجمه دستور شرطی

دستور شرطی age/blood/vagarna به ساختار `if :: ... :: else -> ... fi` در Promela تبدیل می‌شود:

```
1      if
2          :: (condition) ->
3          // then block
4          :: else ->
5          // else block (optional)
6      fi
```

همچنین، اگر شرط شامل عملیات تقسیم باشد، یک شرط اضافی برای بررسی تقسیم بر صفر قبل از شرط اصلی اضافه می‌شود.

۷.۴ متد visitWhileStmt - ترجمه حلقه while

حلقه ta به ساختار `do :: ... :: else -> break od` تبدیل می‌شود:

۱. ایجاد برچسب: یک برچسب یکتا برای شروع حلقه تولید می‌شود (مثلاً `loopStartNo_1`).
۲. افزایش شمارنده: `activeLoopCount` یک واحد افزایش می‌یابد.
۳. ترجمه شرط: شرط حلقه به یک شرط در ساختار `do` تبدیل می‌شود.
۴. مدیریت پرچم‌ها: قبل از بدنه حلقه، پرچم‌های `inLoop` و `exitLoop` تنظیم می‌شوند.
۵. بررسی تقسیم بر صفر: یک شرط اضافی برای تشخیص تقسیم بر صفر قبل از شرط اصلی اضافه می‌شود.
۶. پس از خروج: پس از خروج از حلقه، `activeLoopCount` کاهش یافته و پرچم‌ها به‌روز می‌شوند.

۸.۴ متد visitForStmt - ترجمه حلقه for

حلقه baraye به ساختار زیر در Promela تبدیل می‌شود:

```
۱ // Initialization
۲ activeLoopCount = activeLoopCount + 1;
۳ do
۴ :: (condition) ->
۵ // Set flags
۶ // Loop body
۷ updateLabel:
۸ // Update
۹ :: else -> break
۱۰ od
۱۱ // Reset flags after exit
```

۹.۴ متد visitTryStmt - ترجمه ساختار try-catch

از آنجا که Promela از مفهوم استثنا پشتیبانی نمی‌کند، این ساختار با استفاده از پرچم‌ها و برچسب‌ها شبیه‌سازی می‌شود:

۱. تعریف پرچم: یک پرچم جدید مانند errFlag_1 با مقدار false تعریف می‌شود.
۲. اجرای بلوک try: بدنه emtehan اجرا می‌شود. در صورت بروز خطا (تقسیم بر صفر)، پرچم به true تغییر کرده و به برچسب catchCheck_1 پرش می‌شود.
۳. بررسی پرچم: پس از بلوک try، پرچم بررسی می‌شود و اگر true باشد، بلوک catch اجرا می‌شود.

فصل ۵

توابع کمکی مهم

۱.۵ mapType - نگاشت انواع داده

تبدیل انواع داده Hash به Promela:

```
۱ private String mapType(String hashType) {
۲     switch (hashType) {
۳         case "adad": return "int";
۴         case "boole": return "bool";
۵         default:     return "";
۶     }
۷ }
```

۲.۵ guardDivisionByZero - محافظت از تقسیم بر صفر

این تابع با دریافت لیست مقسوم‌علیه‌های مشکوک، یک ساختار شرطی تولید می‌کند:

```
۱ if
۲ :: (divisor1 == 0 || divisor2 == 0) ->
۳ divByZero = true;
۴ //      try-catch
۵ errFlag = true;
۶ goto catchLabel;
۷ :: else ->
۸ //
۹ fi
```

۳.۵ findZeroDivisors - یافتن مقسوم‌علیه‌های صفر

این تابع با پیمایش بازگشتی درخت عبارت، تمام مقسوم‌علیه‌های عملیات تقسیم یا باقی‌مانده را جمع‌آوری می‌کند. به این صورت که در گره‌های MultiplicativeExprContext (شامل ضرب، تقسیم و باقی‌مانده)، عملگرها را بررسی کرده و اگر عملگر / یا % باشد، عبارت سمت راست را به عنوان یک مقسوم‌علیه مشکوک به لیست اضافه می‌کند. برای سایر گره‌ها، به صورت بازگشتی به فرزندان رفته و جستجو را ادامه می‌دهد. برای مثال، عبارت $(a / b / c) + (e / d)$ خروجی ["b", "c", "d"] را تولید

می‌کند (چون در $a/b/c$ ، مقسوم‌علیه‌های b و c هر دو جمع‌آوری می‌شوند). این لیست سپس برای ساخت شرط بررسی صفر بودن مقسوم‌علیه‌ها استفاده می‌شود. این تابع در متدهای `visitVarDecl`، `visitIfStmt`، `visitAssignmentStmt` و `visitWhileStmt` برای محافظت از تقسیم بر صفر به کار می‌رود.

۴.۵ expandPower - تبدیل توان

از آنجا که Promela از عملگر توان پشتیبانی نمی‌کند، $x^{**}n$ به $x*x*x*...$ (تکرار n بار) تبدیل می‌شود:

```

۱ private String expandPower(String base, int exponent) {
۲     StringBuilder sb = new StringBuilder("(");
۳     for (int i = 0; i < exponent; i++) {
۴         if (i > 0) sb.append(" * ");
۵         sb.append("(").append(base).append(")");
۶     }
۷     sb.append(")");
۸     return sb.toString();
۹ }

```

۵.۵ indent - فرمت‌بندی خروجی

برای افزایش خوانایی کد خروجی، به هر خط ۴ فاصله اضافه می‌کند:

```

۱ private String indent(String code) {
۲     StringBuilder out = new StringBuilder();
۳     for (String line : code.split("\n")) {
۴         if (!line.isBlank()) {
۵             out.append("    ").append(line);
۶         }
۷         out.append("\n");
۸     }
۹     return out.toString();
۱۰ }

```

۶.۵ exitLoopFlags / enterLoopFlags - مدیریت پرچم‌های حلقه

این توابع کد مربوط به تنظیم و بازنشانی پرچم‌های `inLoop` و `exitLoop` را تولید می‌کنند که برای بررسی ویژگی‌های Liveness ضروری هستند.

فصل ۶

نتیجه‌گیری و جمع‌بندی

در این پروژه، یک ترنسلیتور کامل و کارآمد از زبان آموزشی Hash به زبان Promela طراحی و پیاده‌سازی شد. دستاوردهای اصلی این فاز عبارتند از:

۱. **ترجمه دقیق:** تمام ساختارهای زبانی مورد نیاز به درستی به Promela ترجمه می‌شوند.
۲. **مدیریت خواص سیستم:** با افزودن متغیرهای کمکی مانند `divByZero`، `inLoop` و `endReached`، زیرساخت لازم برای بررسی خواص مختلف در فازهای بعدی فراهم شده است.
۳. **پشتیبانی از استثناها:** با وجود اینکه Promela از استثنا پشتیبانی نمی‌کند، با استفاده از تکنیک‌های شبیه‌سازی، ساختار `try-catch` پشتیبانی می‌شود.
۴. **کد خوانا و قابل نگهداری:** استفاده از الگوی `Visitor` و توابع کمکی مناسب، کد را خوانا، ماژولار و به راحتی قابل توسعه کرده است.
۵. **پشتیبانی از عملگرهای پیشرفته:** عملگرهای توان، افزایش/کاهش، و انتساب مرکب به درستی ترجمه می‌شوند.

خروجی تولید شده در این فاز (فایل `output.pml`) به عنوان ورودی فاز دوم (Model Checking) استفاده خواهد شد. در فاز دوم، با استفاده از ابزار SPIN، پنج ویژگی مختلف روی این مدل بررسی می‌شوند تا صحت سیستم تأیید یا رد شود.

• **ویژگی Safety:** جلوگیری از تقسیم بر صفر (`divByZero`)

• **ویژگی Liveness:** خروج از حلقه‌ها (`exitLoop → inLoop`)

• **ویژگی Reachability:** رسیدن به پایان برنامه (`endReached`)

• **ویژگی Freedom Deadlock:** عدم وقوع بن‌بست

• **ویژگی Invariant:** همواره غیرمنفی بودن یک متغیر

این پروژه نشان می‌دهد که چگونه مفاهیم تئوری زبان‌ها و ماشین‌ها (اتوماتا، گرامرها، و معناشناسی) در عمل و برای تأیید صحت نرم‌افزارهای واقعی به کار گرفته می‌شوند. توانایی اثبات ریاضی درستی نرم‌افزارها، یکی از مهارت‌های کلیدی در صنعت نرم‌افزار مدرن است.